

Con el fin de darle mayor modularidad al algoritmo de eliminación en árboles balanceados, se estudiarán dos algoritmos auxiliares. El primero, **Reestructura_izq**, se utiliza cuando la altura de la rama izquierda ha disminuido. El segundo, **Reestructura_der**, se emplea cuando la altura de la rama derecha ha disminuido.

Algoritmo 6.11 Reestructura_izq

Reestructura_izq (NODO, BO)

{Este algoritmo reestructura el árbol cuando la altura de la rama izquierda ha disminuido y el *FE* de NODO es igual a 1. NODO es un parámetro por referencia de tipo puntero. BO es un parámetro de tipo booleano, también por referencia. BO se utiliza para indicar que la altura de la rama izquierda ha disminuido}

{NODO1 y NODO2 son variables auxiliares de tipo puntero}

1. Si (BO = VERDADERO)

entonces

1.1 Si (NODO[^].FE)

= -1: Hacer NODO[^].FE $\leftarrow$ 0

= 0: Hacer NODO[^].FE $\leftarrow$ 1 y BO $\leftarrow$ FALSO

= 1: {Reestructuración del árbol}

Hacer NODO1 $\leftarrow$ NODO[^].DER

1.1.1 Si (NODO1[^].FE $\geq$ 0)

entonces {Rotación DD}

Hacer NODO[^].DER $\leftarrow$ NODO1[^].IZQ y NODO1[^].IZQ $\leftarrow$ NODO

1.1.1.1 Si NODO1[^].FE

= 0: Hacer NODO[^].FE $\leftarrow$ 1, NODO1[^].FE $\leftarrow$ -1 y

BO $\leftarrow$ FALSO

= 1: Hacer NODO[^].FE $\leftarrow$ 0 y NODO1[^].FE $\leftarrow$ 0

1.1.1.2 {Fin del condicional 1.1.1.1}

Hacer NODO $\leftarrow$ NODO1

{Termina la rotación DD}

si no {Rotación DI}

Hacer NODO2 $\leftarrow$ NODO1[^].IZQ, NODO[^].DER $\leftarrow$ NODO2[^].IZQ,

NODO2[^].IZQ $\leftarrow$ NODO, NODO1[^].IZQ $\leftarrow$ NODO2[^].DER y

NODO2[^].DER $\leftarrow$ NODO1

1.1.1.3 Si (NODO2[^].FE = 1)

entonces

Hacer NODO[^].FE $\leftarrow$ -1

si no

Hacer NODO[^].FE $\leftarrow$ 0

1.1.1.4 {Fin del condicional 1.1.1.3}

1.1.1.5 Si (NODO2[^].FE = -1)

entonces

Hacer NODO1[^].FE $\leftarrow$ 1

si no

Hacer NODO1[^].FE $\leftarrow$ 0

- 1.1.1.6 {Fin del condicional 1.1.1.5}
Hacer $NODO \leftarrow NODO2$ y $NODO2.FE \leftarrow 0$
{Termina la rotación *DI*}
- 1.1.2 {Fin del condicional del paso 1.1.1}
- 1.2 {Fin del condicional del paso 1.1}
- 2. {Fin del condicional del paso 1}

Algoritmo 6.12 Reestructura_der

Reestructura_der

{Este algoritmo reestructura el árbol cuando la altura de la rama derecha ha disminuido y el *FE* de *NODO* es igual a -1 . *NODO* es un parámetro, por referencia, de tipo puntero. *BO* es un parámetro de tipo booleano, también por referencia. *BO* se utiliza para indicar que la altura de la rama derecha ha disminuido}
{*NODO1* y *NODO2* son variables auxiliares de tipo puntero}

1. Si (*BO* = VERDADERO) entonces
 - 1.1 Si $NODO.FE$
 - = 1 : Hacer $NODO.FE \leftarrow 0$
 - = 0 : Hacer $NODO.FE \leftarrow -1$ y *BO* $\leftarrow$ FALSO
 - = -1: {Reestructuración del árbol}
Hacer $NODO1 \leftarrow NODO.IZQ$
 - 1.1.1 Si ($NODO1.FE \leq 0$)
entonces {Rotación *II*}
Hacer $NODO.IZQ \leftarrow NODO1.DER$ y $NODO1.DER \leftarrow NODO$
 - 1.1.1.1 Si $NODO1.FE$
 - = 0: Hacer $NODO.FE \leftarrow -1$, $NODO1.FE \leftarrow 1$ y
BO $\leftarrow$ FALSO
 - = -1: Hacer $NODO.FE \leftarrow 0$ y $NODO1.FE \leftarrow 0$
 - 1.1.1.2 {Fin del condicional del paso 1.1.1.1}
Hacer $NODO \leftarrow NODO1$
{Termina la rotación *II*}
 - si no {Rotación *ID*}
Hacer $NODO2 \leftarrow NODO1.DER$, $NODO.IZQ \leftarrow NODO2.DER$,
 $NODO2.DER \leftarrow NODO$, $NODO1.DER \leftarrow NODO2.IZQ$ y
 $NODO2.IZQ \leftarrow NODO1$
 - 1.1.1.3 Si ($NODO2.FE = -1$)
entonces
Hacer $NODO.FE \leftarrow 1$
 - si no
Hacer $NODO.FE \leftarrow 0$
 - 1.1.1.4 {Fin del condicional del paso 1.1.1.3}
 - 1.1.1.5 Si ($NODO2.FE = 1$)
entonces
Hacer $NODO1.FE \leftarrow -1$

```

    si no
        Hacer NODO1^.FE ← 0
    1.1.1.6 {Fin del condicional del paso 1.1.1.5}
        Hacer NODO ← NODO2 y NODO2^.FE ← 0
        {Termina la rotación ID}
    1.1.2 {Fin del condicional del paso 1.1.1}
    1.2 {Fin del condicional del paso 1.1}
    2. {Fin del condicional del paso 1}

```

A continuación se presenta el algoritmo de eliminación en árboles balanceados, el cual hará uso de los previamente explicados.

Algoritmo 6.13 Elimina_balanceado

Elimina_balanceado (NODO, BO, INFOR)

{El algoritmo elimina un elemento en un árbol balanceado. Utiliza dos algoritmos auxiliares Reestructura_izq y Reestructura_der. NODO es un parámetro por referencia de tipo puntero. BO es un parámetro de tipo booleano, también por referencia, y se utiliza para indicar que la altura del árbol ha disminuido, su valor inicial es FALSO. INFOR es un parámetro de tipo entero que contiene la información del elemento que se quiere eliminar}
{OTRO, AUX, AUX1 son variables auxiliares de tipo puntero. BOOL es una variable de tipo booleano}

```

1. Si (NODO ≠ NIL)
    entonces
    1.1 Si (INFOR < NODO^.INFO)
        entonces
            Regresar a Elimina_balanceado con NODO^.IZQ, BO e INFOR
            Llamar al algoritmo Reestructura_izq con NODO y BO
        si no
    1.1.1 Si (INFOR > NODO^.INFO)
        entonces
            Regresar a Elimina_balanceado con NODO^.DER, BO e INFOR
            Llamar al algoritmo Reestructura_der con NODO y BO
        si no
            Hacer OTRO ← NODO y BO ← VERDADERO
    1.1.1.1 Si (OTRO^.DER = NIL)
        entonces
            Hacer NODO ← OTRO^.IZQ
        si no
    1.1.1.1.1 Si (OTRO^.IZQ = NIL)
        entonces
            Hacer NODO ← OTRO^.DER
        si no

```

```

Hacer AUX ← NODO^.IZQ y BOOL ← FALSO
1.1.1.1.1.A Mientras (AUX^.DER ≠ NIL) Repetir
    Hacer AUX1 ← AUX, AUX ← AUX^.DER
    y BOOL ← VERDADERO
1.1.1.1.1.B {Fin del ciclo del paso 1.1.1.1.1.A}
    Hacer NODO^.INFO ← AUX^.INFO y
    OTRO ← AUX
1.1.1.1.1.C Si (BOOL = VERDADERO)
    entonces
        Hacer AUX1^.DER ← AUX^.IZQ
    si no
        Hacer NODO^.IZQ ← AUX^.IZQ
1.1.1.1.1.D {Fin del condicional del paso 1.1.1.1.1.C}
    Llamar al algoritmo Reestructura_der
    con NODO^.IZQ y BO
1.1.1.1.2 {Fin del condicional del paso 1.1.1.1.1}
1.1.1.2 {Fin del condicional del paso 1.1.1.1}
    Quitar (OTRO) {Libera la memoria del nodo}
1.1.2 {Fin del condicional del paso 1.1.1}
1.2 {Fin del condicional del paso 1.1}
    si no
        Escribir “La información no se encuentra en el árbol”
2. {Fin del condicional del paso 1}

```

El análisis matemático de los algoritmos de inserción —Inserta_balanceado— y eliminación —Elimina_balanceado— demuestra que es posible buscar, insertar y eliminar un elemento en un árbol balanceado de n nodos en $O(\log n)$ unidades de tiempo. Por otra parte, diversos análisis demuestran que son más frecuentes las rotaciones en las operaciones de inserción que en las de eliminación, ya que mientras se produce aproximadamente una rotación por cada dos inserciones, se produce una rotación por cada cinco eliminaciones.

Ejemplo 6.22

Supongamos que se desea eliminar la clave 65 del árbol balanceado de la figura 6.34:

FIGURA 6.34
Árbol balanceado.

